

ResearchMind Summarizer: A Locally-Hosted Multimodal Retrieval-Augmented Generation Pipeline

Raghuvardhan Rao Molugu

Research Consultant

raghuvardhan@researchfoundation.in

Abstract—We present the ResearchMind Summarizer, a research-assistant pipeline that ingests heterogeneous source material—plain text, PDF and office documents, audio, video, and spreadsheets—and produces a structured synthesis using a locally hosted large language model (LLM), avoiding third-party cloud inference. Extracted content from each source is chunked, embedded with a sentence-embedding model, and indexed in a local vector store; when the user supplies a research-focus prompt, the most semantically relevant stored passages are retrieved and folded into a single synthesis call, following the retrieval-augmented generation (RAG) pattern. The pipeline auto-detects available GPU hardware and falls back to CPU execution automatically, with no configuration required from the operator. The system is exposed through a browser interface and was validated end-to-end against a live backend and a local LLM.

Index Terms—retrieval-augmented generation, local large language models, multimodal document processing, vector embeddings, automatic speech recognition

I. Introduction

Researchers routinely work with source material spread across incompatible formats: PDF manuscripts, slide decks, recorded lectures, and tabular datasets. Synthesizing this material into a coherent overview is labor-intensive and increasingly delegated to AI systems, most of which route documents through a third-party cloud API—raising privacy, cost, and availability concerns, particularly for unpublished or sensitive research material.

The ResearchMind Summarizer is designed around a different default: the ingestion, embedding, and summarization pipeline runs entirely on local infrastructure, using a locally hosted LLM server (Ollama), a local automatic speech recognition (ASR) engine, and a local embedding model backed by an on-disk vector store. This paper describes that pipeline’s architecture, its GPU-aware compute placement, its browser interface, and an end-to-end validation of the complete stack. A companion paper describes the Similarity Finder, a separate subsystem built on the same local infrastructure that checks an uploaded paper against the published literature for content overlap—the one capability in the broader ResearchMind system that is inherently non-local.

II. Related Work

Retrieval-augmented generation (RAG) conditions a generative language model on passages retrieved from an external corpus at inference time, improving factual grounding relative to parametric generation alone [2]. The Summarizer applies the same principle in a single-session setting: content extracted from a user’s own uploaded files is embedded and indexed, and, when a research-focus prompt is supplied, semantically relevant chunks are retrieved and folded into the final synthesis prompt. The underlying LLM and embedding model are both built on the Transformer architecture [1]; the embedding model follows the Sentence-BERT approach of fine-tuning a Transformer encoder with a siamese/triplet objective so that cosine similarity between sentence embeddings is meaningful [3]. Approximate nearest-neighbor retrieval over the resulting embeddings is performed with a Hierarchical Navigable Small World (HNSW) graph index [5], as implemented by the ChromaDB vector database used in this system. Speech and audio content is transcribed with a Transformer-based ASR model trained on large-scale weakly supervised data [4], executed locally through a CTranslate2-optimized inference engine.

III. System Architecture

Figure 1 shows the end-to-end pipeline. An uploaded file set is routed by file extension into one of five categories—plain text, rich documents, audio, video, and spreadsheets—each handled by a dedicated processor. Text and document extraction uses format-specific libraries (PyMuPDF for PDF, python-docx and python-pptx for Office formats, odfpy for OpenDocument formats); audio and video are transcribed by a locally hosted Whisper model, with video first demultiplexed to a 16 kHz mono waveform; spreadsheets are summarized statistically (column typing, correlation analysis, categorical value counts) before being handed to the LLM for narrative interpretation. Independent files are processed concurrently through a thread pool, since the workload is I/O- and inference-bound rather than CPU-bound.

Every processor’s output is chunked with a fixed character budget and overlap, embedded, and upserted into a persistent, cosine-indexed ChromaDB collection. A summarization stage merges all per-source outputs into a single budgeted context

block; if the user supplies a research-focus prompt, the system additionally retrieves the most semantically relevant stored chunks and folds them into that context before a single call to the local LLM produces the final structured summary (topic, overview, key points, methodology, data insights, research gaps, and keywords).

The pipeline is exposed through a small REST surface: a single synthesis endpoint that accepts a multipart file set and

an optional context string, a health endpoint that reports LLM reachability and the presence of each format-specific extraction dependency, and a model-listing endpoint used by operator tooling. Uploaded files are written to a per-request temporary directory that is removed once processing completes or fails, so no uploaded content persists on disk beyond the lifetime of a single request; only the derived vector-store index persists across requests, and it is reset at the start of each new session.

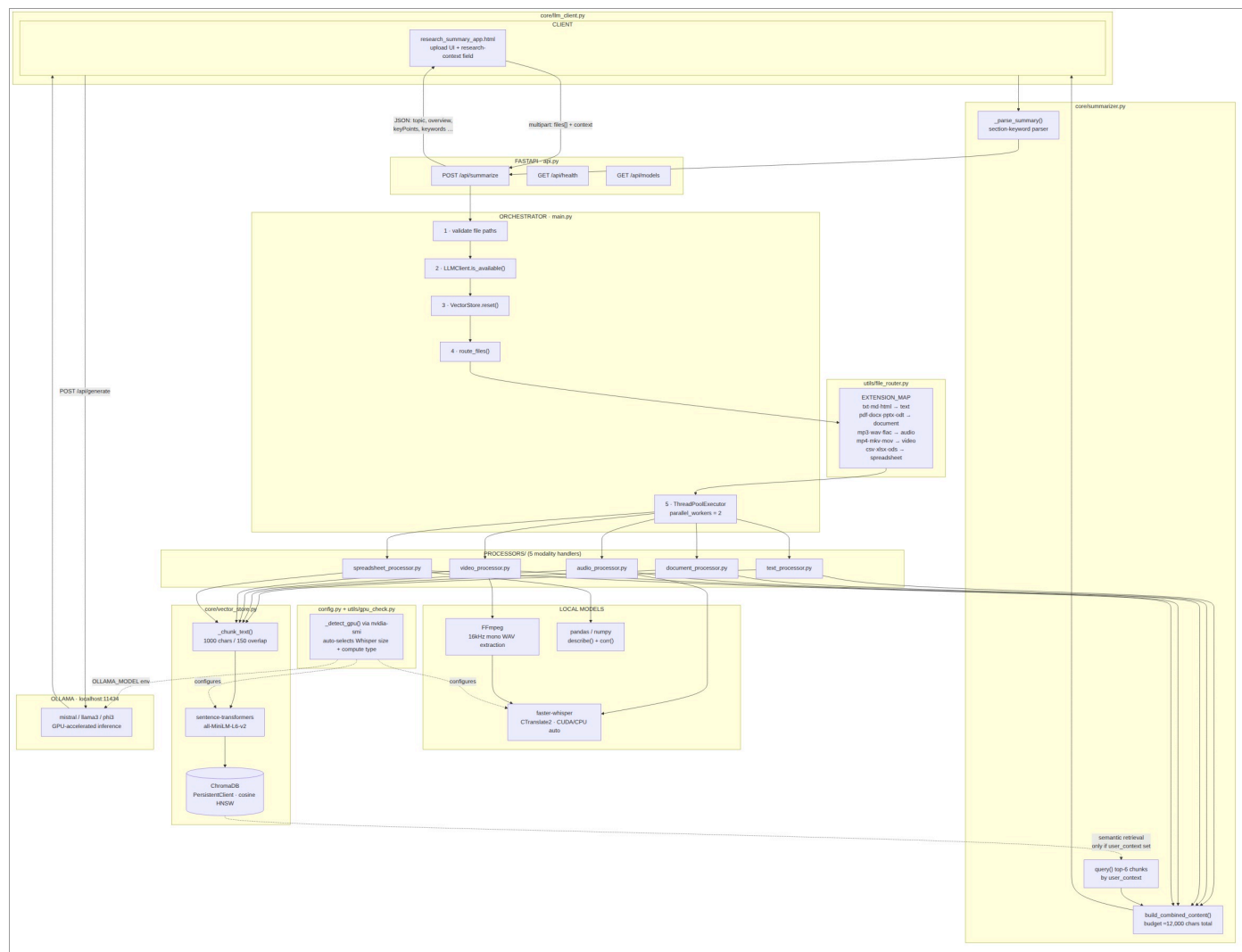


Fig. 1. End-to-end ingestion, retrieval, and synthesis pipeline. Files are routed by modality to dedicated processors, chunked and embedded into a local vector store, and merged into a single LLM synthesis call, with optional semantic retrieval when a research-focus prompt is supplied.

IV. GPU-Aware Compute Placement

Two components in the Summarizer have distinct GPU and CPU execution paths: sentence embeddings (used to populate and query the vector store) and speech transcription (Whisper). LLM inference is delegated to the locally hosted Ollama server, which manages its own GPU usage independently. At process start, the system probes for a usable GPU by querying

the NVIDIA driver and the machine-learning framework’s own device API; when a usable GPU is found, embeddings and transcription default to it for lower latency, and when none is found—or the detected device cannot actually be used—both fall back to CPU execution automatically. This detection and fallback is fully automatic: no environment variable or manual flag needs to be set to choose between GPU and CPU,

and the choice is re-evaluated on every process start rather than cached from a previous run.

We observed that this automatic GPU detection and resource allocation is more reliable when the system runs directly on the host machine or inside a conda-managed environment, rather than inside a plain Python venv virtual environment; under a venv, GPU resource allocation by the underlying machine-learning libraries did not always occur as expected. Deployments intending to rely on GPU acceleration should therefore prefer a conda environment or a bare-metal host installation over a venv-based one.

V. Web-Based User Interface

The Summarizer is exposed through a single-page browser interface that accepts multiple files and an optional research-focus prompt. Uploaded files are classified client-side (document, audio, video, or text) and listed with per-type counts before submission. The result view renders the structured synthesis—main topic, overview, key points, research gaps, and source files analyzed—with copy and Markdown-export actions, and shares its visual design system with a second, Similarity Finder mode described in a companion paper.

VI. Empirical Observation

We validated the pipeline end-to-end against the running backend and a local Ollama instance (7-billion-parameter instruction-tuned model), driven through a headless-browser test of the actual interface rather than by calling internal functions directly. A short research paper was submitted as a plain-text upload; the system returned a coherent structured summary—main topic, a multi-paragraph overview, six key points, and research-gap suggestions—within the same session used to validate the Similarity Finder on identical input, confirming both the document-processing and LLM-integration code paths function correctly. No client-side errors were observed.

VII. Limitations and Future Work

- *Single embedding model.* Retrieval relies on one general-purpose sentence-embedding model; domain-specific or multilingual material may benefit from model selection or ensembling.
- *No streaming.* The final synthesis is returned as a single blocking response rather than streamed incrementally,

which affects perceived latency on longer inputs.

- *Fixed chunking.* Chunk size and overlap are static configuration values rather than adapted to document structure (e.g., section boundaries).
- *Reactive GPU verification.* The detection strategy in Section IV verifies usability at each process start rather than consulting a precompiled compatibility matrix; this trades a small one-time verification cost for not requiring such a matrix to be maintained.

VIII. Conclusion

We described the ResearchMind Summarizer, a locally hosted multimodal research assistant that unifies document, audio, video, and spreadsheet ingestion behind a common retrieval-augmented synthesis pipeline, with automatic GPU-aware compute placement and no operator-facing configuration. The complete system was validated end-to-end through a live browser-driven test against the running backend and a local LLM.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 5998–6008.
- [2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020.
- [3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2019, pp. 3982–3992.
- [4] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” *arXiv preprint arXiv:2212.04356*, 2022.
- [5] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2020.
- [6] Ollama Inc., “Ollama: Get up and running with large language models locally,” 2024. [Online]. Available: <https://ollama.com>
- [7] Chroma Inc., “Chroma: The open-source embedding database,” 2024. [Online]. Available: <https://www.trychroma.com>